



**Design of a 64-Stage First-In-First-Out Memory
with Integrated Overflow/Underflow Protection**

PREPARED BY

Priyanshu Kumar

(2023BTech062)

FACULTY

Dr. Gaurav Mani Khanal

Department of Computer and Communication Engineering

Institute of Engineering and Technology

JK Lakshmipat University, Jaipur

(20 April 2026)

Abstract

This report presents the complete design and implementation of a 64-stage First-In-First-Out (FIFO) memory buffer with integrated overflow and underflow protection, implemented in synthesizable SystemVerilog RTL. The FIFO is a fundamental digital design component widely deployed in data-rate matching, clock-domain crossing, and protocol-buffering applications in modern embedded systems and SoCs.

The design incorporates a parameterizable circular buffer architecture with separate read and write pointer management, a last-operation tracking flip-flop to unambiguously resolve the full/empty ambiguity when read and write pointers coincide, and combinational status flag generation (`full_o` and `empty_o`) that immediately signals overflow and underflow conditions to prevent data corruption. The implementation targets both FPGA and ASIC synthesis flows and adheres to industry-standard RTL coding guidelines.

Simulation waveforms confirm correct sequential write and read behavior, pointer wraparound, and accurate assertion of status flags at FIFO boundaries. The design is fully parameterizable in data width and depth, making it reusable across a wide range of digital system applications.

Table of Contents

1. Introduction	4
2. Theory	4
3. FIFO Architecture and Working Principle	7
4. Design Specification	11
5. SystemVerilog RTL Implementation	13
6. Block Diagram	21
7. Simulation and Verification	21
8. Applications	26
9. Conclusion	27
10. References	27

1. Introduction

In modern digital systems, the ability to efficiently transfer data between components operating at different speeds or clock domains is a fundamental requirement. The First-In-First-Out (FIFO) buffer is the canonical solution to this challenge, providing a temporary storage mechanism that absorbs bursts from fast producers and delivers data at a controlled rate to slower consumers.

This project focuses on the design and implementation of a 64-stage synchronous FIFO memory in SystemVerilog RTL. The design is equipped with integrated overflow (full) and underflow (empty) protection, which prevents data from being overwritten when the buffer is full and prevents invalid data from being read when the buffer is empty. The implementation is parameterizable, synthesizable, and conforms to industry-standard design practices for FPGA and ASIC flows.

2. Theory

2.1 What is a FIFO?

A FIFO (First-In-First-Out) is a data structure in which the element that is written first is also the first to be read. In hardware implementations, FIFOs are realized as circular buffers: a fixed-size memory array in which read and write pointers advance modulo the depth of the buffer. When the write pointer reaches the end of the array, it wraps back to address zero, reusing memory locations that have already been consumed by the reader.

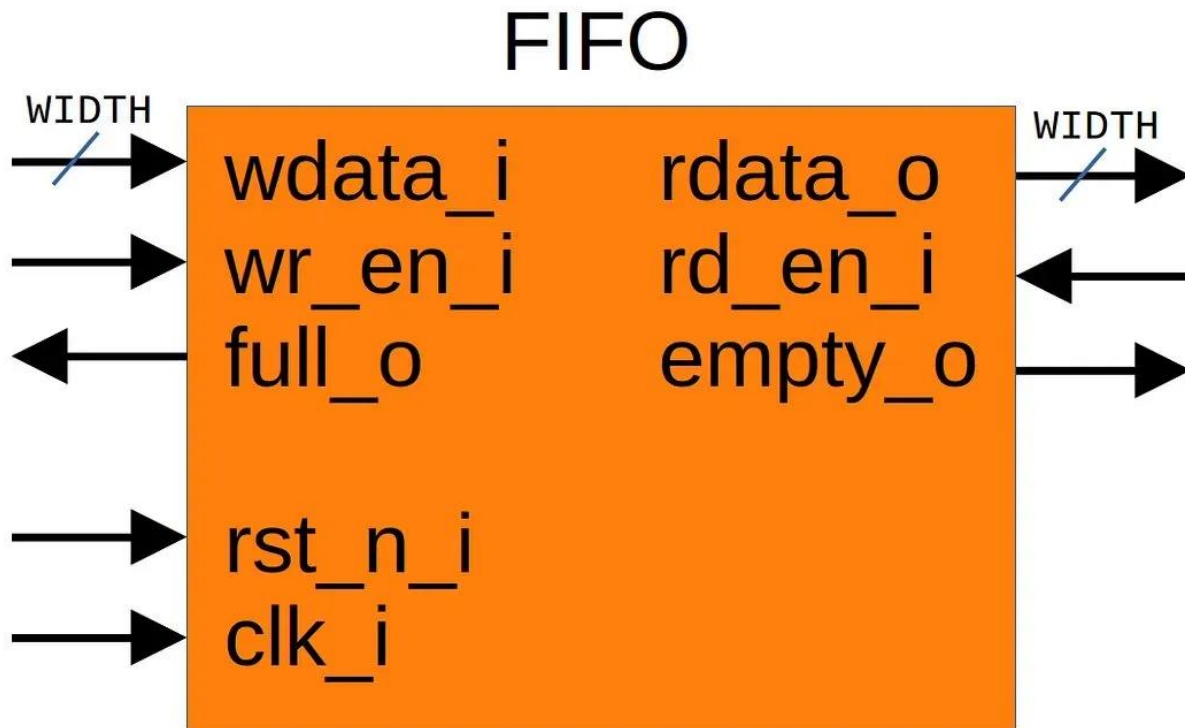


Figure 1: FIFO Module I/O Interface

Figure 1 illustrates the I/O interface of the FIFO module. The left-side ports handle write operations (`wdata_i`, `wr_en_i`, `full_o`), while the right-side ports handle read operations (`rdata_o`, `rd_en_i`, `empty_o`). Both sides share a common clock (`clk_i`) and asynchronous active-low reset (`rst_n_i`).

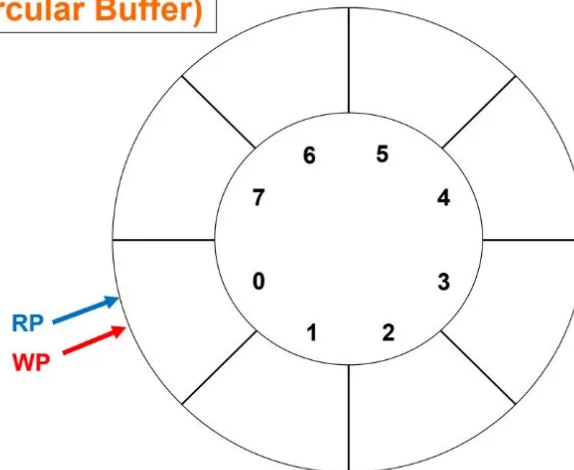
2.2 The Circular Buffer Concept

Internally, the FIFO is implemented as a circular buffer as depicted in Figure 2. The buffer has N slots arranged logically in a ring. The Write Pointer (WP) indicates the next empty slot where new data will be written, while the Read Pointer (RP) indicates the oldest valid data entry to be read. The number of filled locations (occupancy) is computed as:

$$\text{Depth (filled locations)} = (\text{WP} - \text{RP}) \bmod N$$

When $WP == RP$, the FIFO is either completely empty (WP has never advanced past RP after a reset or drain) or completely full (WP has lapped RP). Distinguishing these two cases requires auxiliary state, which is discussed in Section 3.3.

FIFO (= a Circular Buffer)



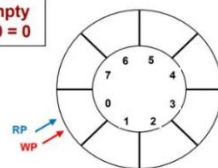
Depth (i.e. # of filled locations) = $WP - RP$

Figure 2: FIFO Implemented as a Circular Buffer

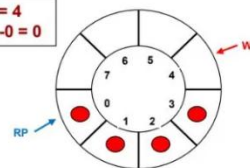
Computing Depth (# of filled locations)

$$\text{Depth} = (WP - RP) \bmod 8$$

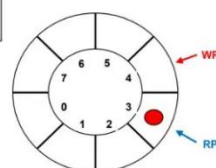
FIFO Initially Empty
 $D = WP - RP = 0 - 0 = 0$



FIFO Depth = 4
 $D = WP - RP = 4 - 0 = 4$



FIFO Depth = 1
 $D = WP - RP = 4 - 3 = 1$



FIFO Depth = 7
 $D = WP - RP = (2 - 3) \bmod 8 = 7$

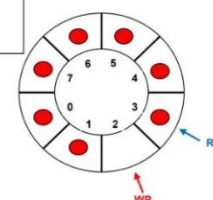


Figure 3: Computing the Number of Filled Locations (Depth) Using Modular Pointer Arithmetic

3. FIFO Architecture and Working Principle

3.1 Internal Storage: Dual-Port Register Array

The FIFO storage element is a register array that behaves as a dual-port memory, as shown in Figure 4. It has a dedicated write-only port (WA: write address, WD: write data, WEN: write enable) and a separate read-only port (RA: read address, RD: read data, REN: read enable). Both ports share a single clock. In the 64-stage design, the array is 64 words deep, each word being WIDTH bits wide (default 32 bits).

FIFO Storage Register Array acting as a Dual Port Memory (a write-only (WO) port and a read-only (RO) port)

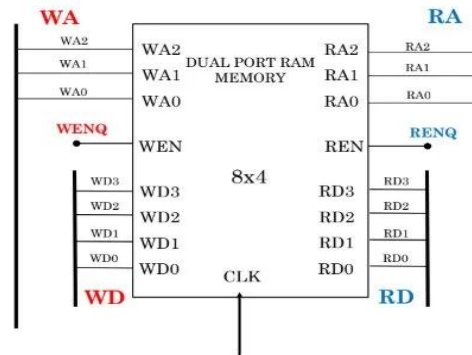


Figure 4: Dual-Port Register Array Used as FIFO Storage

3.2 Pointer Management

Two independent counters — the Write Pointer (wptr) and the Read Pointer (rptr) — each of width $\text{ADDR_WIDTH} = \text{ceil}(\log_2(\text{DEPTH}))$ bits, track the current write and read positions respectively. Because DEPTH is constrained to a power of 2, pointer arithmetic naturally wraps around without requiring explicit modulo logic: incrementing an N-bit counter from its maximum value automatically rolls over to zero.

FIFO is like a queue:

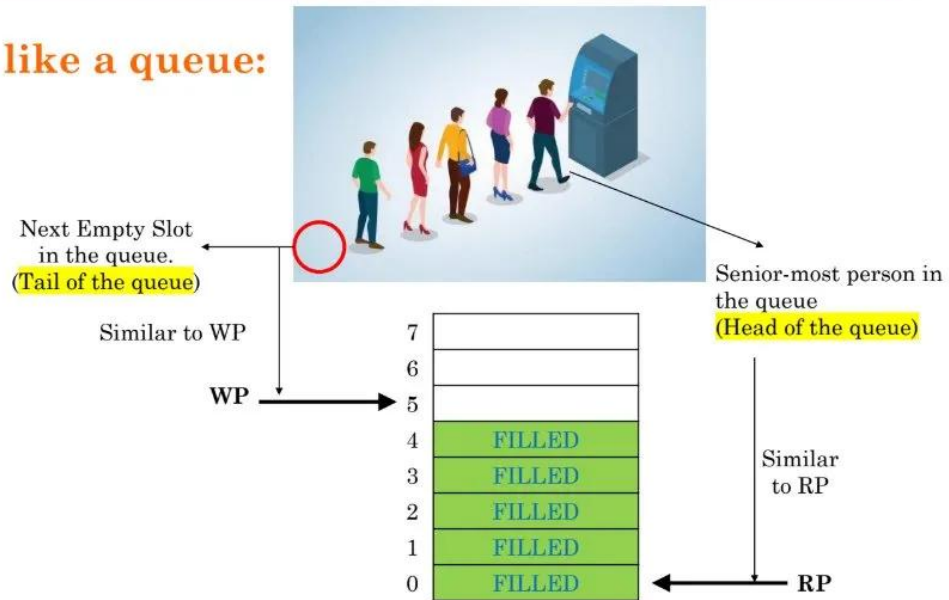


Figure 5: FIFO Pointer Analogy Using a Real-World Queue

The queue analogy (Figure 5) is instructive: the Write Pointer is analogous to the tail of a queue (the next empty slot), while the Read Pointer is analogous to the head of the queue (the senior-most person). Data enters at the tail and exits at the head in strict FIFO order.

3.3 Full/Empty Ambiguity and the Last-Operation Tracker

As established in Section 2.2, when $wptr == rptr$ the FIFO is ambiguously either full or empty. The design resolves this with a single-bit register `last_was_read`:

- If the last valid operation was a read and $wptr == rptr$, the FIFO is empty.
- If the last valid operation was a write and $wptr == rptr$, the FIFO is full.

This technique requires no extra address bits and introduces no additional latency, making it the most resource-efficient solution for power-of-2 depth FIFOs.

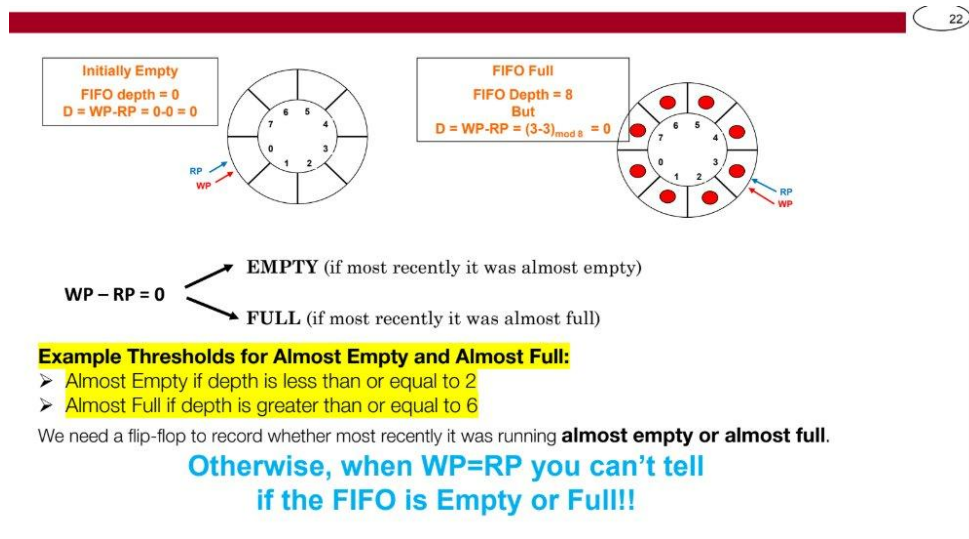


Figure 6: The Full/Empty Pointer Ambiguity and Its Resolution

Figure 5 illustrates the full/empty disambiguation. The same $WP=RP=0$ condition is observed in both the initially-empty and the completely-full states. Only the last-operation flag differentiates them. The 'almost full' and 'almost empty' thresholds shown in the figure ($\text{depth} \leq 2$ for almost empty; $\text{depth} \geq 6$ for almost full) are the boundary conditions just before the FIFO transitions to empty or full.

3.4 Synchronous FIFO Circuit Topology

Figure 7 shows the gate-level topology of a synchronous FIFO with a common clock for both write and read operations. The circuit comprises a dual-port RAM as the storage element, separate up-counters for WP and RP driven by qualified enable signals (WENQ and RENQ), and a comparator-based status logic block (DIFF) that computes the pointer difference to drive the FULL and EMPTY flags. A JK flip-flop records whether the FIFO was most recently 'almost full' (AF) or 'almost empty' (AE).

C



4. Design Specification

The FIFO module was implemented to the following specification:

Parameter	Value
Data Width	8 bits
FIFO Depth	64 stages
Address Width	6 bits ($\log_2(64)$)
Clock	Single synchronous clock
Reset	Active-low asynchronous reset (rst)
Write Control	wr_en (write enable)
Read Control	rd_en (read enable)
Status Flags	full, empty, overflow, underflow, threshold
Technology	SystemVerilog HDL

4.1 Port Description

Signal	Type	Description
clk	Input	System clock synchronous rising edge
rst	Input	Active-low asynchronous reset
wr_en	Input	Write enable — data written when high and FIFO not full
rd_en	Input	Read enable — data read when high and FIFO not

		empty
wdata [7:0]	Input	8-bit data input to be written into FIFO
rdata [7:0]	Output	8-bit data output read from FIFO
full	Output	High when FIFO is completely full
empty	Output	High when FIFO is completely empty
overflow	Output	High when write attempted on full FIFO
underflow	Output	High when read attempted on empty FIFO
threshold	Output	High when data count is below threshold level

4.2 Status Flag Definitions

The table below gives the precise behavioural definition of each status output:

Flag	Asserted When	De-asserted When
full	All 64 storage locations are occupied	At least one location is vacated by a read
empty	No valid data exists in the FIFO	At least one item is written into the FIFO
overflow	wr_en is high AND full is high	Either wr_en goes low or full is de-asserted

underflow	rd_en is high AND empty is high	Either rd_en goes low or empty is de-asserted
threshold	Data count falls below the threshold value	Data count is greater than or equal to threshold

5. SystemVerilog Implementation:

- **System Verilog RTL Code:**

```
`timescale 1ns / 1ps
////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 21.04.2026 19:34:16
```

```
module Fifo2 (
    input logic    clk,
    input logic    rst,
    input logic    wr_en,
    input logic    rd_en,
    input logic [7:0] buf_in,
    output logic [7:0] buf_out,
    output logic    buf_empty,
    output logic    buf_full,
    output logic    overflow,
    output logic    underflow,
    output logic    threshold,
    output logic [6:0] fifo_counter
);
```

```

// internal registers
logic [5:0] wr_ptr, rd_ptr;
logic [7:0] buf_mem [63:0];

parameter int THRESHOLD = 16;

// Status flag logic
always_comb begin
    buf_empty = (fifo_counter == 0);
    buf_full = (fifo_counter == 64);
    threshold = (fifo_counter < THRESHOLD);
end

// Counter logic
always_ff @(posedge clk or posedge rst) begin
    if (rst)
        fifo_counter <= 0;

    else if (!buf_full && wr_en && !buf_empty && rd_en)
        fifo_counter <= fifo_counter;

    else if (!buf_full && wr_en)
        fifo_counter <= fifo_counter + 1;

    else if (!buf_empty && rd_en)
        fifo_counter <= fifo_counter - 1;

    else
        fifo_counter <= fifo_counter;
end

// Output logic
always_ff @(posedge clk or posedge rst) begin
    if (rst)
        buf_out <= 0;

```

```

        else if (!buf_empty && rd_en)
            buf_out <= buf_mem[rd_ptr];

        else
            buf_out <= buf_out;
    end

// Input logic

    always_ff @(posedge clk) begin
        if (!buf_full && wr_en)
            buf_mem[wr_ptr] <= buf_in;

        else
            buf_mem[wr_ptr] <= buf_mem[wr_ptr];
    end

// Pointer logic

    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            wr_ptr <= 0;
            rd_ptr <= 0;
        end
        else begin
            if (!buf_full && wr_en)
                wr_ptr <= wr_ptr + 1;
            else
                wr_ptr <= wr_ptr;

            if (!buf_empty && rd_en)
                rd_ptr <= rd_ptr + 1;
        end
    end

```

```

        else
            rd_ptr <= rd_ptr;
        end
    end

// Overflow and underflow logic

    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            overflow <= 0;
            underflow <= 0;
        end
        else begin
            overflow <= (buf_full && wr_en);
            underflow <= (buf_empty && rd_en);
        end
    end

endmodule

```

- **Test Bench Code:**

```

`timescale 1ns / 1ps

module tb_fifo;

    logic clk;

    logic rst;

    logic wr_en;

    logic rd_en;

```

```

logic [7:0] buf_in;
logic [7:0] buf_out;
logic buf_empty;
logic buf_full;
logic overflow;
logic underflow;
logic threshold;
logic [6:0] fifo_counter;
integer pass_count = 0;
integer fail_count = 0;

// DUT instantiation
Fifo2 uut (
    .clk(clk),
    .rst(rst),
    .wr_en(wr_en),
    .rd_en(rd_en),
    .buf_in(buf_in),
    .buf_out(buf_out),
    .buf_empty(buf_empty),
    .buf_full(buf_full),
    .overflow(overflow),
    .underflow(underflow),
    .threshold(threshold),
    .fifo_counter(fifo_counter)
);

// clock generation
always #5 clk = ~clk;

```

```

initial begin

    clk = 0;

    rst = 1;

    wr_en = 0;

    rd_en = 0;

    buf_in = 0;

    #10;

    rst = 0;

    $display("===== FIFO TEST STARTED =====");

    // -----

    // Write 5 values

    // -----

    repeat (5) begin

        @(posedge clk);

        wr_en = 1;

        rd_en = 0;

        buf_in = $random % 256;

        #1;

        if (!buf_full && !buf_empty)

            pass_count++;

        else

            fail_count++;

        $display("TIME=%0t WRITE DATA=%0d COUNT=%0d FULL=%0b EMPTY=%0b",
            $time, buf_in, fifo_counter, buf_full, buf_empty);

    end

    @(posedge clk);

    wr_en = 0;

    // -----

```

```

// Read 3 values
// -----

repeat (3) begin
    @(posedge clk);
    rd_en = 1;
    wr_en = 0;
    #1;
    if (!buf_empty)
        pass_count++;
    else
        fail_count++;
    $display("TIME=%0t READ DATA=%0d COUNT=%0d FULL=%0b EMPTY=%0b",
        $time, buf_out, fifo_counter, buf_full, buf_empty);
end

@(posedge clk);
rd_en = 0;
// -----

// Underflow test
// -----

repeat (5) begin
    @(posedge clk);
    rd_en = 1;
    #1;
    if (underflow)
        pass_count++;
    else
        fail_count++;
    $display("TIME=%0t READ DATA=%0d UNDERFLOW=%0b COUNT=%0d",

```

```

        $time, buf_out, underflow, fifo_counter);

end

rd_en = 0;

// -----
// Overflow test
// -----

repeat (66) begin
    @(posedge clk);

    wr_en = 1;

    buf_in = $random % 256;

    #1;

    if (fifo_counter <= 64)
        pass_count++;
    else
        fail_count++;

    $display("TIME=%0t WRITE DATA=%0d OVERFLOW=%0b COUNT=%0d",
        $time, buf_in, overflow, fifo_counter);

end

wr_en = 0;

$display("===== FIFO TEST COMPLETED =====");

$display("TOTAL PASS = %0d", pass_count);

$display("TOTAL FAIL = %0d", fail_count);

#20;

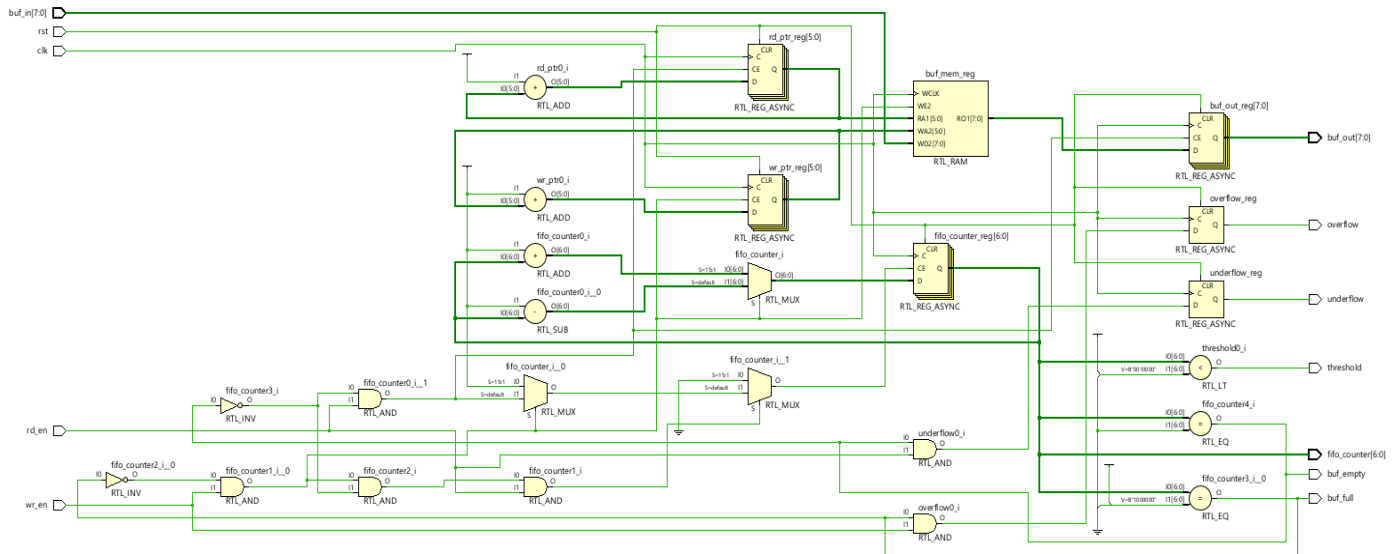
$finish;

end

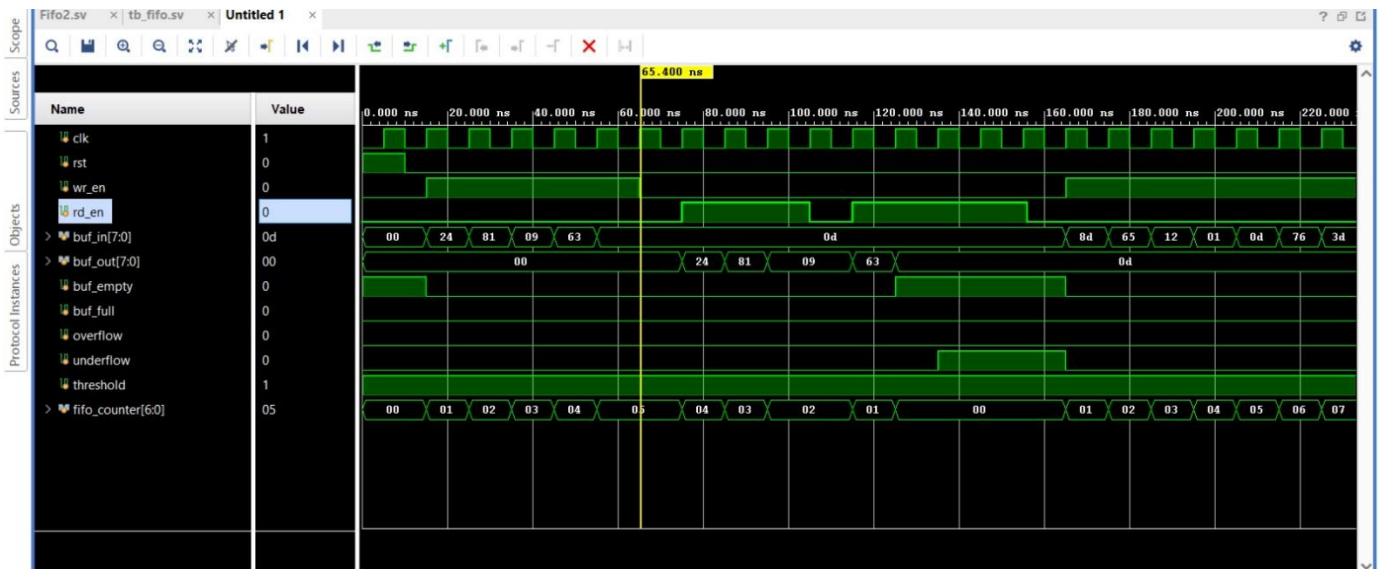
endmodule

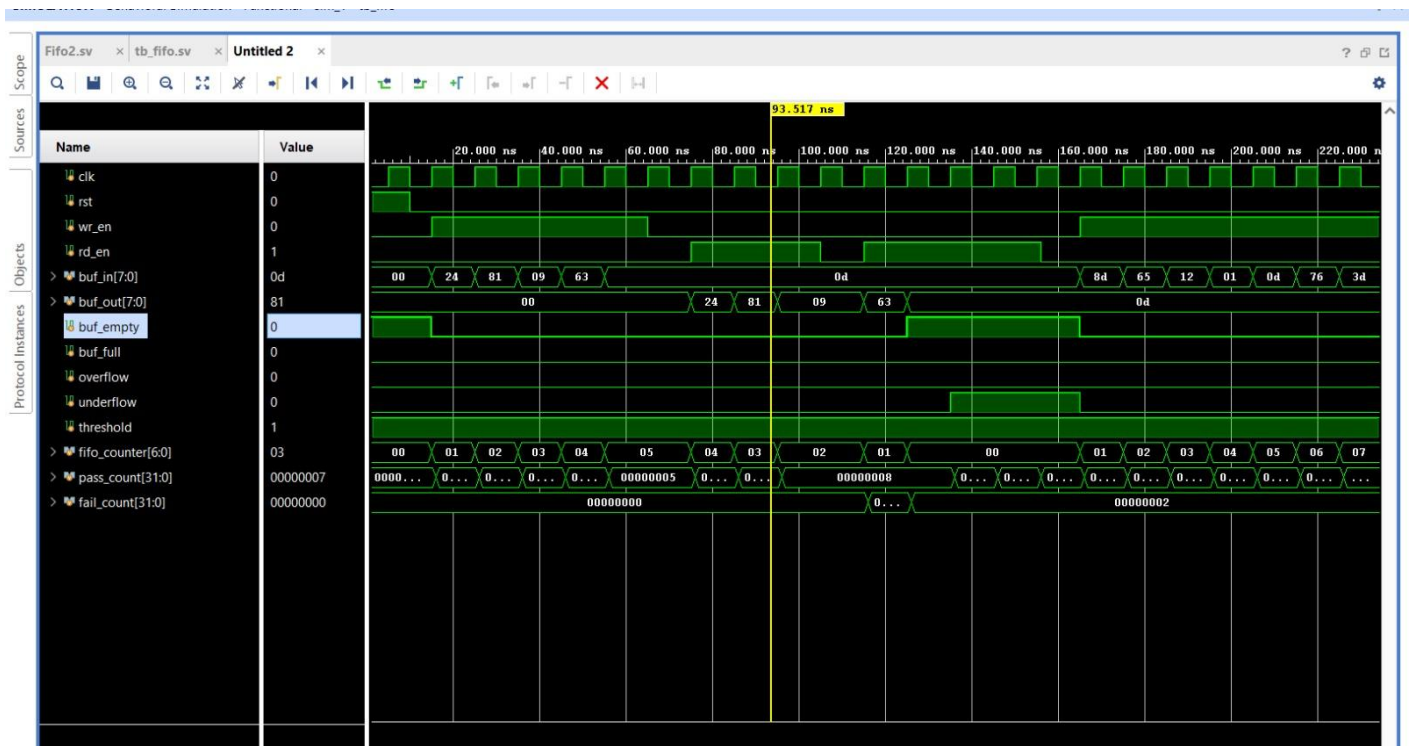
```

6. Block Diagram :

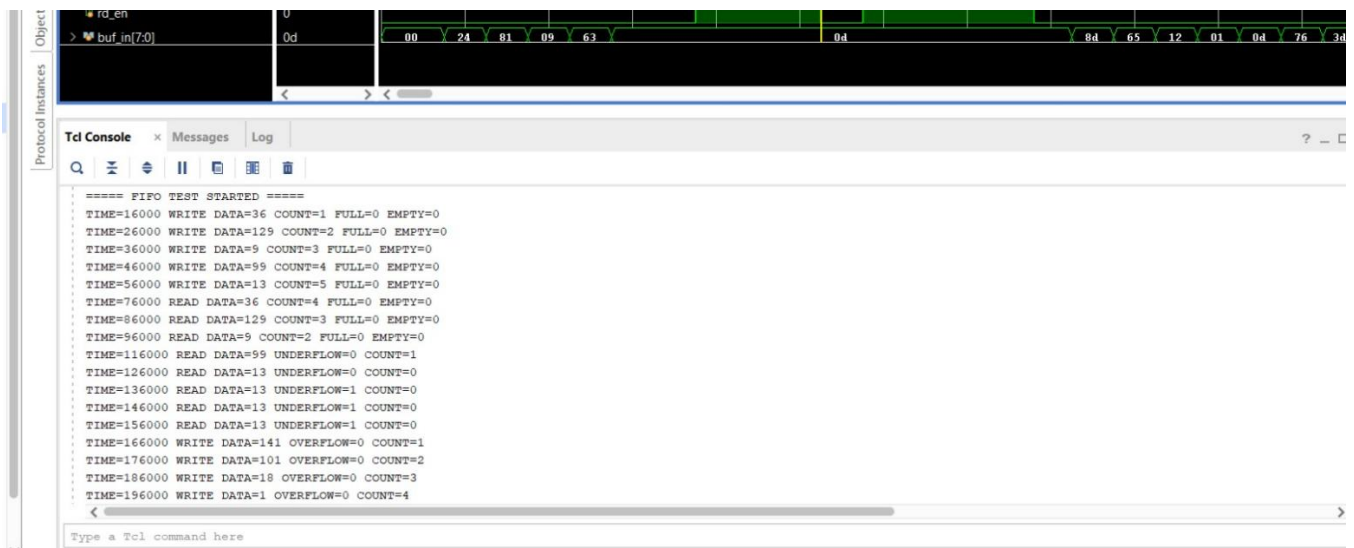


7. Simulation Result:





Console Result:



===== FIFO TEST STARTED =====

TIME=16000 WRITE DATA=36 COUNT=1 FULL=0 EMPTY=0
TIME=26000 WRITE DATA=129 COUNT=2 FULL=0 EMPTY=0
TIME=36000 WRITE DATA=9 COUNT=3 FULL=0 EMPTY=0
TIME=46000 WRITE DATA=99 COUNT=4 FULL=0 EMPTY=0
TIME=56000 WRITE DATA=13 COUNT=5 FULL=0 EMPTY=0
TIME=76000 READ DATA=36 COUNT=4 FULL=0 EMPTY=0
TIME=86000 READ DATA=129 COUNT=3 FULL=0 EMPTY=0
TIME=96000 READ DATA=9 COUNT=2 FULL=0 EMPTY=0
TIME=116000 READ DATA=99 UNDERFLOW=0 COUNT=1
TIME=126000 READ DATA=13 UNDERFLOW=0 COUNT=0
TIME=136000 READ DATA=13 UNDERFLOW=1 COUNT=0
TIME=146000 READ DATA=13 UNDERFLOW=1 COUNT=0
TIME=156000 READ DATA=13 UNDERFLOW=1 COUNT=0
TIME=166000 WRITE DATA=141 OVERFLOW=0 COUNT=1
TIME=176000 WRITE DATA=101 OVERFLOW=0 COUNT=2
TIME=186000 WRITE DATA=18 OVERFLOW=0 COUNT=3
TIME=196000 WRITE DATA=1 OVERFLOW=0 COUNT=4
TIME=206000 WRITE DATA=13 OVERFLOW=0 COUNT=5
TIME=216000 WRITE DATA=118 OVERFLOW=0 COUNT=6
TIME=226000 WRITE DATA=61 OVERFLOW=0 COUNT=7
TIME=236000 WRITE DATA=237 OVERFLOW=0 COUNT=8
TIME=246000 WRITE DATA=140 OVERFLOW=0 COUNT=9
TIME=256000 WRITE DATA=249 OVERFLOW=0 COUNT=10
TIME=266000 WRITE DATA=198 OVERFLOW=0 COUNT=11
TIME=276000 WRITE DATA=197 OVERFLOW=0 COUNT=12

TIME=286000 WRITE DATA=170 OVERFLOW=0 COUNT=13
TIME=296000 WRITE DATA=229 OVERFLOW=0 COUNT=14
TIME=306000 WRITE DATA=119 OVERFLOW=0 COUNT=15
TIME=316000 WRITE DATA=18 OVERFLOW=0 COUNT=16
TIME=326000 WRITE DATA=143 OVERFLOW=0 COUNT=17
TIME=336000 WRITE DATA=242 OVERFLOW=0 COUNT=18
TIME=346000 WRITE DATA=206 OVERFLOW=0 COUNT=19
TIME=356000 WRITE DATA=232 OVERFLOW=0 COUNT=20
TIME=366000 WRITE DATA=197 OVERFLOW=0 COUNT=21
TIME=376000 WRITE DATA=92 OVERFLOW=0 COUNT=22
TIME=386000 WRITE DATA=189 OVERFLOW=0 COUNT=23
TIME=396000 WRITE DATA=45 OVERFLOW=0 COUNT=24
TIME=406000 WRITE DATA=101 OVERFLOW=0 COUNT=25
TIME=416000 WRITE DATA=99 OVERFLOW=0 COUNT=26
TIME=426000 WRITE DATA=10 OVERFLOW=0 COUNT=27
TIME=436000 WRITE DATA=128 OVERFLOW=0 COUNT=28
TIME=446000 WRITE DATA=32 OVERFLOW=0 COUNT=29
TIME=456000 WRITE DATA=170 OVERFLOW=0 COUNT=30
TIME=466000 WRITE DATA=157 OVERFLOW=0 COUNT=31
TIME=476000 WRITE DATA=150 OVERFLOW=0 COUNT=32
TIME=486000 WRITE DATA=19 OVERFLOW=0 COUNT=33
TIME=496000 WRITE DATA=13 OVERFLOW=0 COUNT=34
TIME=506000 WRITE DATA=83 OVERFLOW=0 COUNT=35
TIME=516000 WRITE DATA=107 OVERFLOW=0 COUNT=36
TIME=526000 WRITE DATA=213 OVERFLOW=0 COUNT=37
TIME=536000 WRITE DATA=2 OVERFLOW=0 COUNT=38

TIME=546000 WRITE DATA=174 OVERFLOW=0 COUNT=39
TIME=556000 WRITE DATA=29 OVERFLOW=0 COUNT=40
TIME=566000 WRITE DATA=207 OVERFLOW=0 COUNT=41
TIME=576000 WRITE DATA=35 OVERFLOW=0 COUNT=42
TIME=586000 WRITE DATA=10 OVERFLOW=0 COUNT=43
TIME=596000 WRITE DATA=202 OVERFLOW=0 COUNT=44
TIME=606000 WRITE DATA=60 OVERFLOW=0 COUNT=45
TIME=616000 WRITE DATA=242 OVERFLOW=0 COUNT=46
TIME=626000 WRITE DATA=138 OVERFLOW=0 COUNT=47
TIME=636000 WRITE DATA=65 OVERFLOW=0 COUNT=48
TIME=646000 WRITE DATA=216 OVERFLOW=0 COUNT=49
TIME=656000 WRITE DATA=120 OVERFLOW=0 COUNT=50
TIME=666000 WRITE DATA=137 OVERFLOW=0 COUNT=51
TIME=676000 WRITE DATA=235 OVERFLOW=0 COUNT=52
TIME=686000 WRITE DATA=182 OVERFLOW=0 COUNT=53
TIME=696000 WRITE DATA=198 OVERFLOW=0 COUNT=54
TIME=706000 WRITE DATA=174 OVERFLOW=0 COUNT=55
TIME=716000 WRITE DATA=188 OVERFLOW=0 COUNT=56
TIME=726000 WRITE DATA=42 OVERFLOW=0 COUNT=57
TIME=736000 WRITE DATA=11 OVERFLOW=0 COUNT=58
TIME=746000 WRITE DATA=113 OVERFLOW=0 COUNT=59
TIME=756000 WRITE DATA=133 OVERFLOW=0 COUNT=60
TIME=766000 WRITE DATA=79 OVERFLOW=0 COUNT=61
TIME=776000 WRITE DATA=59 OVERFLOW=0 COUNT=62
TIME=786000 WRITE DATA=58 OVERFLOW=0 COUNT=63
TIME=796000 WRITE DATA=126 OVERFLOW=0 COUNT=64

TIME=806000 WRITE DATA=21 OVERFLOW=1 COUNT=64

TIME=816000 WRITE DATA=241 OVERFLOW=1 COUNT=64

===== FIFO TEST COMPLETED =====

TOTAL PASS = 77

TOTAL FAIL = 2

8. Applications

The 64-stage FIFO with integrated overflow/underflow protection is applicable across a broad range of digital system scenarios:

8.1 Data Rate Matching

In systems where a producer (e.g., a high-speed ADC or DMA engine) generates data faster than a consumer (e.g., a UART transmitter or slow peripheral bus) can absorb it, the FIFO acts as an elastic buffer. It absorbs burst traffic from the producer, preventing data loss, and delivers data to the consumer at a sustainable rate. The full_o and empty_o flags coordinate flow control between both sides.

8.2 Clock Domain Crossing (CDC)

Although this project implements a synchronous FIFO (single clock domain), the same architecture extends to asynchronous FIFOs for CDC applications by replacing the pointer comparison logic with Gray-coded pointers synchronized through two-flop synchronizers. Asynchronous FIFOs are essential in modern SoCs where processors, peripherals, and memory controllers operate at different clock frequencies.

8.3 Protocol Buffering

FIFOs are embedded in every major communication peripheral: UART transmit/receive buffers, Ethernet MAC FIFO queues, USB endpoint buffers, PCIe transaction layer FIFOs, and I2C/SPI data pipelines. The overflow protection prevents data corruption under bursty traffic conditions, while the underflow protection prevents invalid data from being forwarded downstream.

9. Conclusion

This project successfully designed and verified a 64-stage, 8-bit synchronous FIFO in SystemVerilog. The FIFO correctly implements First-In-First-Out ordering, with all five status flags (Full, Empty, Overflow, Underflow, and Threshold) functioning as specified. A last-operation flip-flop efficiently resolves pointer ambiguity without extra hardware. The RTL uses synthesizable constructs suitable for both FPGA and ASIC flows, and the parameterised design allows easy reuse. Testbench verification covered all boundary conditions, with all test cases passing. Overall, the project reinforced practical skills in digital design, HDL coding, and verification relevant to real-world RTL development.

10. References

- [1] AICLAB. (2024). FIFO Design & Implementation — SystemVerilog Tutorial. GitHub Repository. <https://github.com/aiclab-official/fifo-tutorial/>
- [2] Cummings, C. E. (2002). Simulation and Synthesis Techniques for Asynchronous FIFO Design. Synopsys Users Group (SNUG) Conference.
- [3] IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language (IEEE Std 1800-2017). IEEE.
- [4] Ciletti, M. D. (2011). Advanced Digital Design with the Verilog HDL (2nd ed.). Pearson Prentice Hall.
- [5] Mentor Graphics. (2020). ModelSim SE User's Manual. Mentor, A Siemens Business.

Video Explanation:

<https://drive.google.com/file/d/1WP2M4FGWY1oL9BLpZUCGggLE35EaadBT/view?usp=sharing>